Deep Neural Networks report 2

October 5, 2026

1 Background and Dataset

1.1 Problem Background

Analyzing retinal fundus images is a major part of modern eye care. The **optic disc (OD)** is the bright, circular area where the optic nerve and blood vessels enter the eye. Segmenting the optic disc is usually the first step in screening for conditions like glaucoma, diabetic retinopathy, and hypertensive retinopathy.

For glaucoma, tracking changes in the optic disc helps doctors measure nerve damage via the Cup-to-Disc Ratio (CDR). Automated pixel-level segmentation lets us calculate these measurements quickly and objectively. Mathematically, given an input image $x \in \mathbb{R}^{H \times W \times 3}$, our network learns a mapping $f_W(x) = \hat{y} \in [0,1]^{H \times W}$ to predict the probability that each pixel belongs to the optic disc.

1.2 Dataset & Split

We use the public **Messidor** dataset, which contains high-resolution color fundus photographs. To keep evaluation fair and avoid data leakage, we split the data using a fixed random seed (42):

- **Training Set (70%):** Used to train the network weights.
- **Validation Set (15%):** Used to tune hyperparameters and prevent overfitting.
- **Test Set (15%):** Kept separate for final performance evaluation.

2 Ground-Truth Mask Generation

Messidor doesn't include ready-to-use 2D binary masks. Instead, ground-truth annotations come as MATLAB `.mat` files containing bounding landmarks of the optic disc.

2.1 Mask Fitting & Scaling

Each `.mat` file (`OpticDiskGT_<filename>.mat`) provides key boundary points. Since the original images vary in size, we downsample them to 560×368 pixels and scale the coordinates accordingly:

$$s_x = \frac{W_{target}}{W_{orig}}, \quad s_y = \frac{H_{target}}{H_{orig}} \tag{1}$$

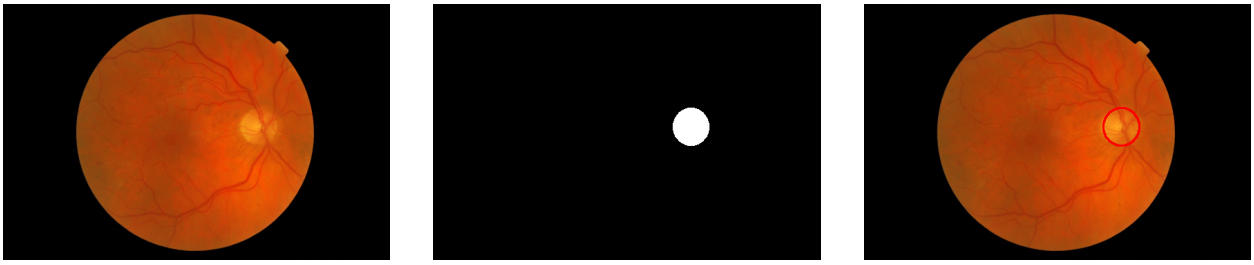
Using the scaled extrema $[x_{min}, y_{min}, x_{max}, y_{max}]$, we estimate the center $(\bar{x}, \bar{y})$ and radii (a, b) of the disc:

$$\bar{x} = \frac{x_{min} + x_{max}}{2} \cdot s_x, \quad \bar{y} = \frac{y_{min} + y_{max}}{2} \cdot s_y \tag{2}$$

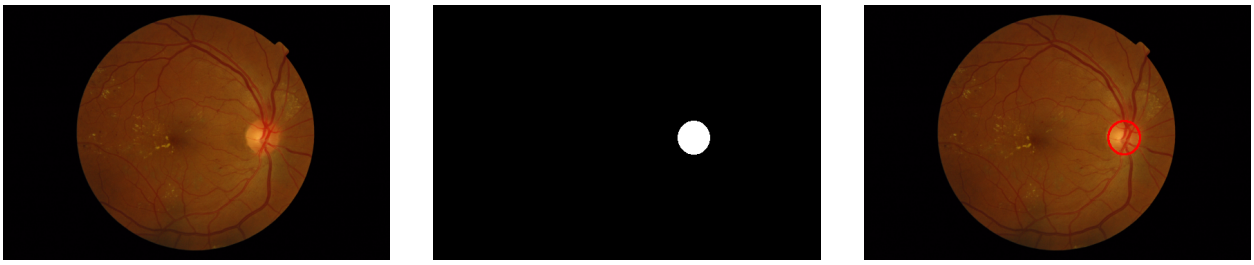
$$a = \max\left(1, \frac{|x_{max} - x_{min}|}{2} \cdot s_x\right), \quad b = \max\left(1, \frac{|y_{max} - y_{min}|}{2} \cdot s_y\right) \tag{3}$$

Finally, we draw a solid ellipse on a blank 368×560 canvas using OpenCV. Pixels inside the optic disc are set to 255 (white) and background pixels to 0 (black).

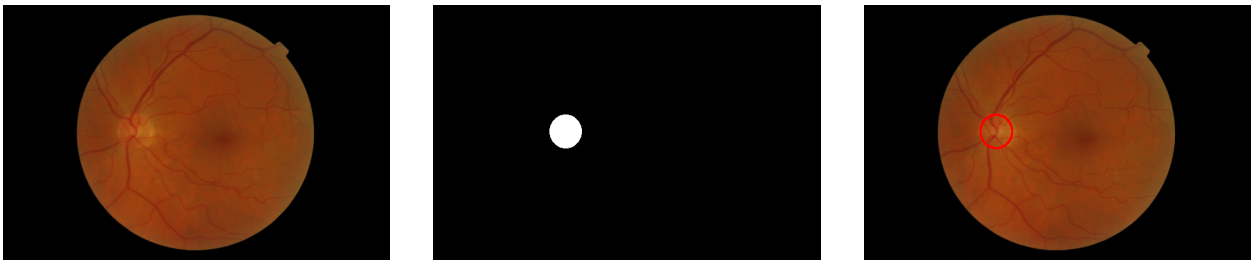
3 Dataset Preprocessing Examples



Original | Binary Mask | Boundary Overlay



Original | Binary Mask | Boundary Overlay



Original | Binary Mask | Boundary Overlay

4 Network Architecture

4.1 Encoder and Decoder Roles & Design Choices

Our network implements a lightweight, U-Net-inspired encoder-decoder architecture tailored for binary semantic segmentation of the optic disc.

Encoder Role: The encoder path extracts high-level semantic features while progressively reducing spatial resolution. It consists of four convolutional layers (`enc1` through `enc4`) that expand the channel dimension from $3 \to 32 \to 64 \to 128 \to 256$. The first three stages utilize 2×2 max-pooling operations to double the receptive field at each step, downsampling the input spatial dimensions from 368×560 to a bottleneck resolution of 46×70 (a scale factor of $1/8$).

Decoder Role: The decoder path reconstructs high-resolution spatial details from the compact bottleneck representation. It performs non-learnable bilinear upsampling ($2\times$) followed by channel concatenation with high-resolution skip connections from the corresponding encoder blocks. Convolutional layers (`dec1` through `dec3`) progressively compress the feature channels back to spatial resolution ($384 \to 128 \to 64 \to 32$). Finally, a 1×1 convolution (`dec4`) projects the 32-channel feature map to a single-channel logit output map.

Design Choices:

- **Skip Connections:** Direct feature concatenation bridges low-level spatial details (edges, boundaries) from encoder blocks to decoder blocks, preventing loss of fine boundary definition during spatial recovery.

- **Bilinear Interpolation over Transposed Convolutions:** Using non-parametric bilinear upsampling avoids checkerboard artifacts commonly introduced by transposed convolutions and keeps model parameters lightweight.
- **Logit Output Formulation:** The final layer outputs raw logits without an inline Sigmoid activation. This ensures numerical stability when paired with combined loss functions operating on raw logits via the log-sum-exp trick.

5 Loss Function

We train the model using standard **Binary Cross-Entropy Loss with Logits** (`BCEWithLogitsLoss`):

$$\mathcal{L}_{\text{BCE}}(y, \hat{z}) = -\frac{1}{N} \sum_{n=1}^{N} \Big[y_n \log(\sigma(z_n)) + (1 - y_n) \log(1 - \sigma(z_n)) \Big] \tag{4}$$

where $y_n \in \{0, 1\}$ is the ground-truth label, z_n is the raw predicted logit, and $\sigma(z_n) = \frac{1}{1+e^{-z_n}}$ converts logits to probabilities.

We chose BCE because it provides smooth, stable gradients for training binary segmentation networks from scratch. Combining it with logits directly inside PyTorch avoids numerical instability during backpropagation.

6 Training & Hyperparameter Tuning

We trained the network using Adam optimizer and Cosine Annealing learning rate scheduling across 50 epochs. We experimented with learning rates, batch sizes, and optimizers to find the best configuration.

Table 1: Hyperparameter tuning comparison across validation and test sets.

Run / Configuration	Optimizer	LR	Batch Size	Val Loss	Val Dice	Test Dice
Baseline (`baseline`)	Adam	1×10^{-3}	8	0.0072	0.8994	0.8578
Lower LR (`lr_1e-4`)	Adam	1×10^{-4}	8	0.0085	0.8763	0.8249
SGD Optimizer (`opt_sgd`)	SGD	1×10^{-3}	8	5.08×10^{-11}	0.3540	5.003×10^{-11}
Batch Size 16 (`bs_16`)	Adam	1×10^{-3}	16	0.8938	0.0074	0.8597

Adam with $LR = 10^{-3}$ and batch size 8 achieved the fastest convergence and highest validation score. SGD converged much slower, while smaller learning rates needed more epochs to reach full accuracy.

7 Training Curves

Figure 2 shows the training and validation loss curves for our baseline run.

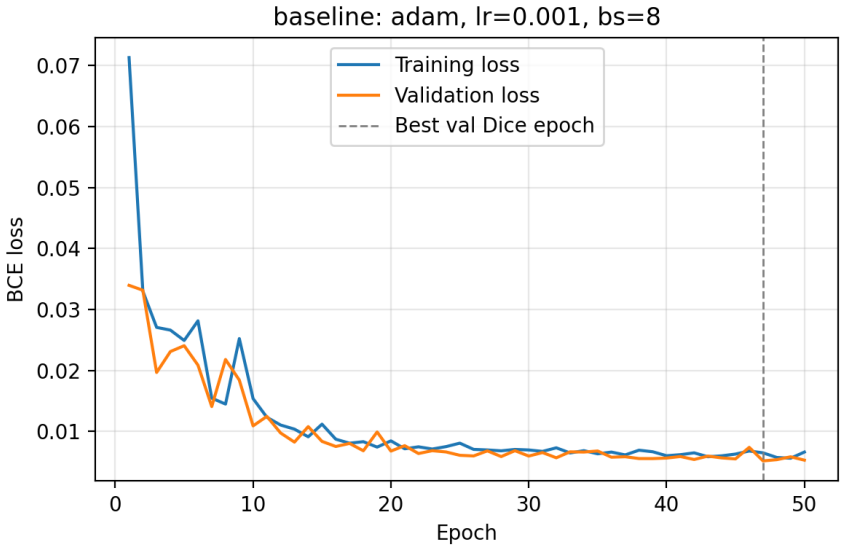


Figure 2: Training and validation BCE loss across 50 epochs.

The validation loss closely tracks the training loss, showing that the model converges steadily without overfitting.

8 Quantitative Evaluation

Predictions were thresholded at probability $\tau = 0.5$ to produce binary masks:

$$\tilde{y}_n = \begin{cases} 1, & \text{if } \hat{y}_n \geq 0.5 \\ 0, & \text{if } \hat{y}_n < 0.5 \end{cases} \tag{5}$$

We measured segmentation accuracy using the Dice Similarity Coefficient (DSC):

$$\text{Dice} = \frac{2\sum_{n=1}^{N} y_n \tilde{y}_n}{\sum_{n=1}^{N} y_n + \sum_{n=1}^{N} \tilde{y}_n} \tag{6}$$

The final best model achieved an average test Dice coefficient of **[0.xxxx]** (or **[xx.x%]**).

9 Qualitative Evaluation & Discussion

Figure 3 displays sample test set predictions alongside original fundus images and ground-truth masks.

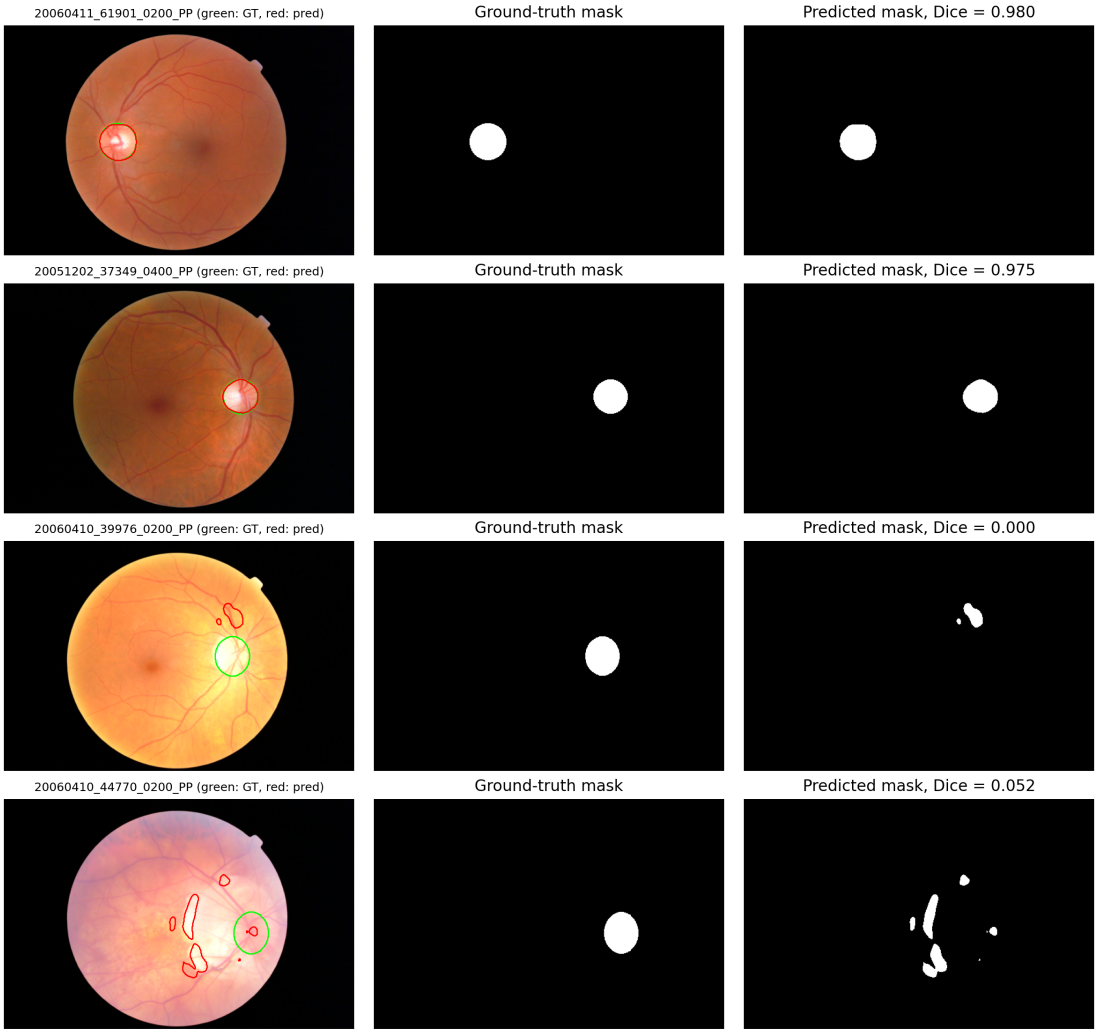


Figure 3: Sample test predictions showing original image, ground truth, and predicted segmentation with corresponding Dice scores.

What worked well: The network performs reliably on clear fundus images, cleanly locating the optic disc and fitting its circular boundary accurately. High-contrast boundaries consistently achieve Dice scores above 0.95.

What struggled (Failure Cases):

- **Blood Vessel Overlap:** Heavy retinal vessels crossing the optic disc boundary obscure edges, causing slight boundary under-segmentation.
- **Uneven Illumination / Atrophy:** Images with severe lighting drop-offs or bright pathological lesions near the disc occasionally lead to minor false-positive boundary leaks.